Jobsheet 1

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom

Disusun oleh:
Mukhammad Raffi Zabra

T1 – 1G

254107020059

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2026-2027

1. Langkah Percobaan

1.1 Struktur Project



Gambar 1. Struktur project

Penjelasan Struktur Folder:

- Workspace & Package: Seluruh *source code* dan dokumen dikelola di dalam folder utama PBO dan dikelompokkan secara rapi ke dalam satu *package* Java bernama P1.
- Ekstensi File (.java & .class): Terlihat bahwa setiap class memiliki dua wujud file. File .java merupakan kode sumber yang ditulis manual, sedangkan file .class merupakan *bytecode* hasil kompilasi otomatis dari Java yang siap dieksekusi.
- File Percobaan Panduan: File Bike.java, RoadBike.java, dan BikeDemo.java merupakan kelompok file dari percobaan dasar pengenalan class dan inheritance sesuai instruksi awal jobsheet.
- File Tugas Praktikum: Terdiri dari class mandiri (laptop.java dan tws.java), superclass (charger.java), dan subclass turunannya (chargerHP.java dan chargerLaptop.java).
- Main Class: Terdapat file perangkatDemo.java yang memuat metode main, bertindak sebagai *driver class* untuk menginstansiasi dan mengeksekusi semua objek dari tugas praktikum.

1.2 Percobaan Praktikum (Jobsheet)

Percobaan 1: Dasar Class Bike dan Driver BikeDemo

- **Code Class Bike.java**

```java
public class Bike{
    private String brand;
    private int speed;
    private int gear=1;

    private final int[] GEAR_SPEED_LIMITS = {5, 10, 25, 30, 40, 60};
    public void setBrand(String brandName){
        brand = brandName;
    }
    public void gearChanges(int gearValue){
        if (gearValue < 1 || gearValue > 6) {
            System.out.println(x: "Invalid gear value. Gear must be between 1 and 6.");
        }
        else{
            gear = gearValue;
        }
    }
    public int speedAcceleration(int increment) {
        speed += increment;
        if (speed > GEAR_SPEED_LIMITS[gear - 1]) {
            speed = GEAR_SPEED_LIMITS[gear - 1];
        }
        return speed;
    }
    public int speedDeceleration(int decrement) {
        speed -= decrement;
        if (speed < 0) {
            speed = 0;
        }
        return speed;
    }
    public void printInfo(){
        System.out.println(x: "damn");
        System.out.println("Brand : " + brand);
        System.out.println("Speed : " + speed);
        System.out.println("Gear : " + gear);
    }
}
```

Gambar 2. Code percobaan Class Bike.java

- **Code Class BikeDemo.java**

```java
public class BikeDemo {
    Run | Debug
    public static void main(String[] args){
        Bike mountainBike1 = new Bike();
        Bike mountainBike2 = new Bike();

        mountainBike1.setBrand(brandName: "Trek");
        mountainBike1.speedAcceleration(increment: 10);
        mountainBike1.gearChanges(gearValue: 2);
        mountainBike1.printInfo();

        mountainBike2.setBrand(brandName: "Giant");
        mountainBike2.speedAcceleration(increment: 20);
        mountainBike2.gearChanges(gearValue: 3);
        mountainBike2.printInfo();
    }
}
```

Gambar 3. Code percobaan Class BikeDemo.java

- **Hasil**

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PBO> javac P1/BikeDemo.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PBO> java P1.BikeDemo
Brand : Trek
Speed : 5
Gear : 2
Brand : Giant
Speed : 5
Gear : 3
```

Gambar 4. Hasil percobaan

- **Penjelasan:** Program ini mendefinisikan class Bike dengan atribut status brand, speed, dan gear menggunakan hak akses private (enkapsulasi). Kecepatan sepeda dibatasi oleh array GEAR_SPEED_LIMITS.
- **Konsep OOP/Java:** Penerapan enkapsulasi, fungsi *setter*, serta validasi logika batasan kecepatan pada method speedAcceleration.
- **Analisis Output:** Output Speed pada mountainBike1 dan mountainBike2 bernilai 5 karena akselerasi dijalankan saat gir masih bernilai default (Gear 1) sebelum proses pindah gir dieksekusi.

Percobaan 2: Pewarisan Class RoadBike

- **Code modifikasi Class Bike.java**



```
package P1;

public class Bike{
    private String brand;
    private int speed;
    private int gear=1;

    private final int[] GEAR_SPEED_LIMITS = {5, 10, 25, 30, 40, 60};
    public void setBrand(String brandName){
        brand = brandName;
    }
    public void gearChanges(int gearValue){
        if (gearValue < 1 || gearValue > 6) {
            System.out.println("Invalid gear value. Gear must be between 1 and 6.");
        }
        else{
            gear = gearValue;
        }
    }
    public int speedAcceleration(int increment) {
        speed += increment;
        if (speed > GEAR_SPEED_LIMITS[gear - 1]) {
            speed = GEAR_SPEED_LIMITS[gear - 1];
        }
        return speed;
    }
    public int speedDeceleration(int decrement) {
        speed -= decrement;
        if (speed < 0) {
            speed = 0;
        }
        return speed;
    }
    public void printInfo(){
        System.out.println("Brand : " + brand);
        System.out.println("Speed : " + speed);
        System.out.println("Gear : " + gear);
    }
}
```

Gambar 5. Code modifikasi percobaan Class Bike.java

- **Code modifikasi Class BikeDemo.java**



```
package P1;

public class BikeDemo {
    Run | Debug
    public static void main(String[] args){
        Bike mountainBike1 = new Bike();
        Bike mountainBike2 = new Bike();
        RoadBike roadBike1 = new RoadBike();

        mountainBike1.setBrand(brandName: "Trek");
        mountainBike1.speedAcceleration(increment: 10);
        mountainBike1.gearChanges(gearValue: 2);
        mountainBike1.printInfo();

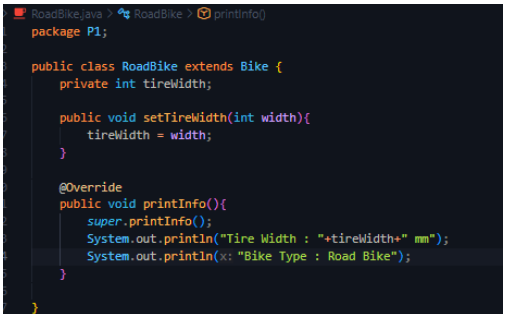
        mountainBike2.setBrand(brandName: "Giant");
        mountainBike2.speedAcceleration(increment: 20);
        mountainBike2.gearChanges(gearValue: 3);
        mountainBike2.printInfo();

        roadBike1.setBrand(brandName: "Specialized");
        roadBike1.setTireWidth(width: 25);
        roadBike1.speedAcceleration(increment: 15);
        roadBike1.gearChanges(gearValue: 4);
        roadBike1.printInfo();

    }
}
```

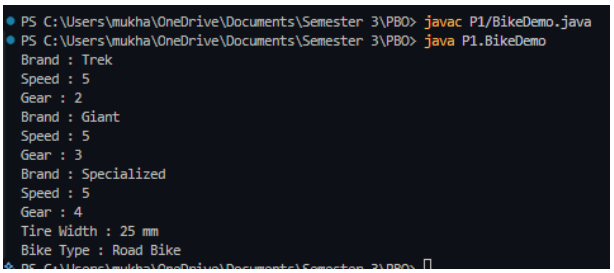
Gambar 6. Code modifikasi percobaan Class BikeDemo.java

- **Tambah Class RoadBike.java**



Gambar 7. Tambah Class percobaan RoadBike.java

- **Hasil**



Gambar 8. Hasil dari percobaan

- **Penjelasan:** Menambahkan class RoadBike sebagai turunan dari class Bike dengan kata kunci extends. Class turunan ini menambahkan atribut spesifik tireWidth dan meng-override method printInfo() dengan memanfaatkan super.printInfo() untuk memanggil kembali pencetakan data induknya.

1.3. Pertanyaan

1. Jelaskan perbedaan antara object dengan class!
Jawaban :

- Class adalah template yang mendefinisikan struktur data (atribut) dan perilaku (metode) yang akan dimiliki oleh suatu entitas. Contoh: public class Bike, bikedemo, roadbike.

- Object adalah wujud nyata dari suatu kelas yang memiliki alokasi memori serta nilai data tersendiri saat program berjalan. Contoh: mountainBike1 dan roadBike1.

2. Jelaskan alasan gear dan brand dapat menjadi atribut dari object Bike!
Jawaban :

- Karena brand dan gear merepresentasikan karakteristik, properti, atau status (*state*) nyata pada setiap sepeda. dalam konsep OOP, data atau variabel yang menggambarkan kondisi dari entitas tersebut dimodelkan sebagai atribut.

3. Sebutkan salah satu kelebihan utama dari pemrograman berorientasi objek dibandingkan dengan pemrograman prosedural!
Jawaban :

- Mendukung Reusabilitas Kode melalui Pewarisan : Memungkinkan kita membuat kelas baru dengan mewarisi kode dari kelas yang sudah ada tanpa harus menulis ulang logika dasarnya (seperti kelas RoadBike yang mewarisi fungsi dari kelas Bike).

4. Apakah diperbolehkan melakukan pendefinisian dua buah atribut dalam satu baris kode seperti “public String nama, alamat;”?
Jawaban :

- Boleh, Java mengizinkan deklarasi beberapa variabel bertipe data sama dalam satu baris yang dipisahkan koma. Namun, praktik terbaik dalam penulisan kode Java tetap menyarankan penulisan satu baris untuk satu variabel agar lebih mudah dibaca, dikelola, dan diberi komentar.

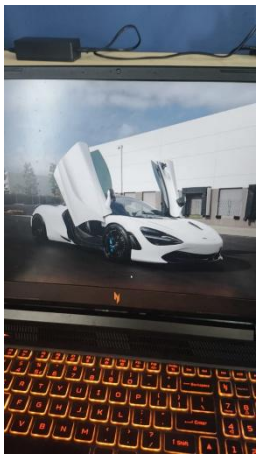
5. Pada class RoadBike, jelaskan alasan atribut brand, speed, dan gear tidak lagi ditulis di dalam class tersebut!
Jawaban :

- Karena kelas RoadBike merupakan turunan dari kelas Bike menggunakan kata kunci extends. Semua atribut dan metode publik dari parent class (Bike) secara otomatis diwarisi ke RoadBike, sehingga RoadBike cukup mendefinisikan atribut atau metode tambahan khusus yang belum ada pada kelas induknya (seperti tireWidth)

2. Tugas Praktikum

a. Foto 4 buah objek di sekitar kalian dengan 2 objek di antaranya merupakan objek yang mengandung konsep pewarisan (inheritance), contoh: kulkas, kursi, meja ruang tamu, meja belajar sehingga diketahui meja ruang tamu dan meja belajar mewarisi objek meja!
Jawaban :

- Laptop



Gambar 9. Object Laptop

- TWS



Gambar 10. Object TWS

- Charger Hp



Gambar 11. Object Charger Hp

- Charger Laptop



Gambar 12. Object Charger Laptop

b. Lakukan pengamatan terhadap 4 objek tersebut untuk menentukan atribut dan methodnya!
Jawaban :
1. Objek: Laptop
- Atribut: brand (String), ramSize (int)
- Method: powerOn(), openApp(String appName), printInfo()

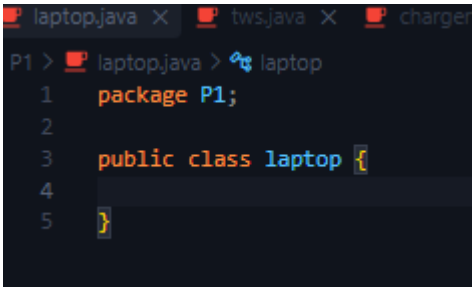
2. Objek: TWS
- Atribut: brand (String), batteryLevel (int)
- Method: connectBluetooth(), playMusic(), printInfo()

3. Objek: Charger HP (*Subclass* dari *Class Induk* Charger)
- Atribut Warisan (Charger): brand (String), wattage (int)
- Atribut Khusus: portType (String), isFastCharging (boolean)
- Method Warisan (Charger): plugIn(), unplug()
- Method Khusus/Override: chargePhone(), activateFastCharge(), printInfo()

4. Objek: Charger Laptop (*Subclass* dari *Class Induk* Charger)
- Atribut Warisan (Charger): brand (String), wattage (int)
- Atribut Khusus: adapterSize (String), connectorType (String)
- Method Warisan (Charger): plugIn(), unplug()
- Method Khusus/Override: chargeLaptop(), checkGrounding(), printInfo()

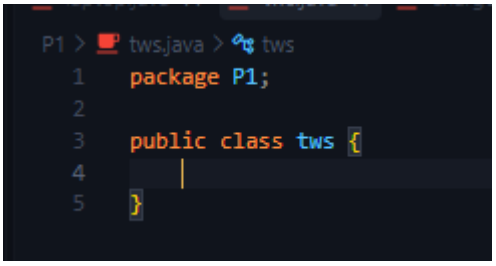
c. Berdasarkan 4 buah objek tersebut, buat class nya dalam Bahasa pemrograman Java!
Jawaban :

- Class Laptop.java Dibuat untuk memodelkan entitas laptop dengan properti teknis dan fungsinya saat digunakan.



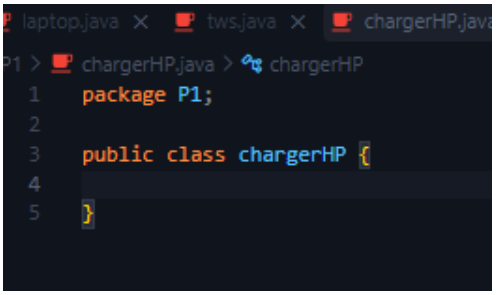
Gambar 13. Buat Class Laptop.java

- Class Tws.java Dibuat untuk memodelkan earphone nirkabel beserta status daya dan interaksi audionya.



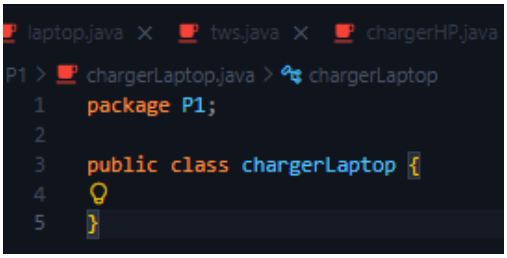
Gambar 14. Buat Class tws.java

- Class Charger HP Dibuat untuk memodelkan adaptor pengisi daya ponsel cerdas.



Gambar 15. Buat class chargerHP.java

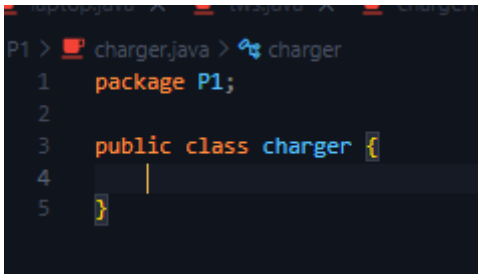
- Class Charger Laptop Dibuat untuk memodelkan adaptor pengisi daya perangkat laptop.



Gambar 16. Buat class chargerLaptop.java

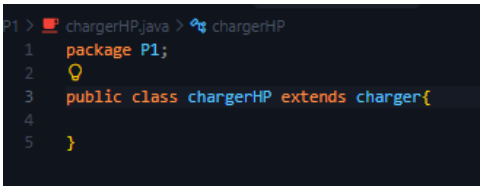
d. Perlu diperhatikan bahwa terdapat dua class hasil pewarisan sehingga perlu menambah satu class baru sebagai class yang mewarisi dua class tersebut!
Jawaban :

- Class Charger Ditambahkan sebagai kelas induk umum karena chargerHp dan chargerLaptop berbagi karakteristik yang identik (memiliki daya dan colokan ke sumber listrik).



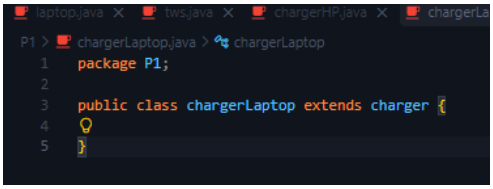
Gambar 17. Buat class induk charger.java

- Class ChargerHp Menambahkan deklarasi extends Charger agar otomatis mewarisi sifat dasar charger umum.



Gambar 18. Mendeklarasikan extends charger di chargerHp.java

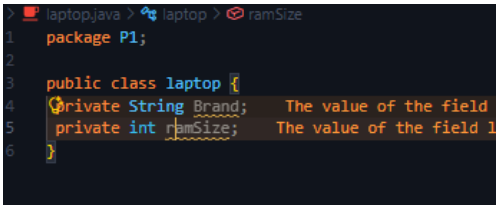
- Class charger Laptop Menambahkan deklarasi extends Charger agar menggunakan kembali logika dasar charger tanpa menulis ulang kode dari nol.



Gambar 19. Mendeklarasikan extends charger di chargerLaptop.java

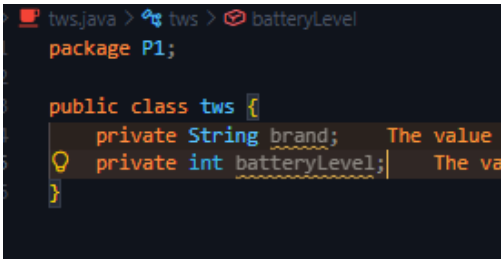
e. Tambahkan dua atribut untuk setiap class!
Jawaban :

- Class Laptop : brand untuk menyimpan identitas merk laptop, ramSize menyimpan kapasitas memori kerja laptop



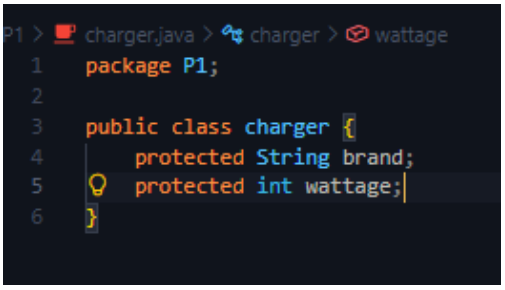
Gambar 20. Menambahkan 2 atribut di class laptop

- Class tws : brand menyimpan identitas merek TWS, batteryLevel Menyimpan angka persentase daya baterai yang tersisa.



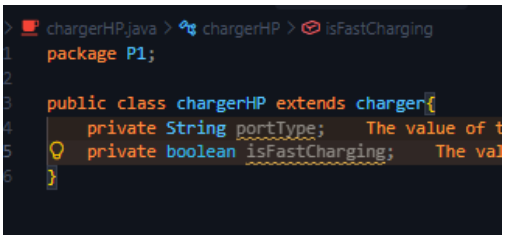
Gambar 21. Menambahkan 2 atribut di class tws

- Class Charger ini sebagai induk : brand menyimpan nama produsen charger, wattage Menyimpan besaran daya listrik maksimum.



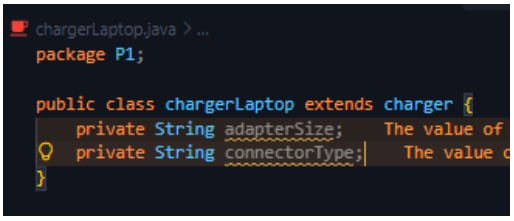
Gambar 21. Menambahkan 2 atribut di class charger

- Class Charger Hp turunan : portType Menyimpan jenis port kabel data/daya, isFastCharging Menyimpan nilai boolean untuk ketersediaan fitur pengisian cepat



Gambar 22. Menambahkan 2 atribut di class charger Hp

- Class Charger Laptop turunan : adapterSize Menyimpan ukuran dimensi fisik bodi adaptor, connectorType Menyimpan tipe colokan daya ujung laptop.



Gambar 23. Menambahkan 2 atribut di class charger Laptop

f. Tambahkan tiga method untuk setiap class termasuk method cetak informasi!
Jawaban :

- Class Laptop : powerOn(): Menyalakan sistem laptop, openApp Membuka perangkat lunak tertentu sesuai argumen teks, printInfo() Mencetak seluruh data atribut brand dan ramSize.



Gambar 24. Menambahkan 3 method di class Laptop

- Class Tws : connectBluetooth() Mengaktifkan koneksi nirkabel ke sumber audio, playMusic() Memulai pemutaran audio/musi, printInfo() Menampilkan sisa persentase baterai dan merek TWS.



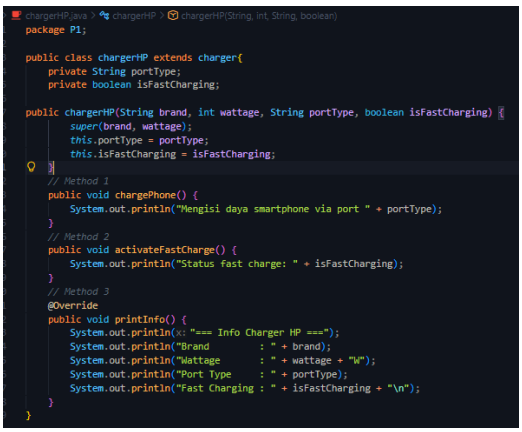
Gambar 25. Menambahkan 3 method di class tws

- Class Charger sebagai induk : plugIn() Menghubungkan steker adaptor ke sumber listrik, unplug() Memutus hubungan charger dari stopkontak, printInfo() Menampilkan spesifikasi merek dan daya dasar (Watt).



Gambar 26. Menambahkan 3 method di class charger

- Class Charger Hp sebagai turunan : chargePhone() Menghantarkan arus listrik ke baterai HP, activateFastCharge() Memeriksa dan menjalankan pengisian arus berkecepatan tinggi, printInfo() Mencetak data lengkap charger beserta port dan dukungan *fast charge.*



```
chargerHP.java > chargerHP > chargerHP(String, int, String, boolean)
package P1;

public class chargerHP extends charger{
    private String portType;
    private boolean isFastCharging;

public chargerHP(String brand, int wattage, String portType, boolean isFastCharging) {
        super(brand, wattage);
        this.portType = portType;
        this.isFastCharging = isFastCharging;
    }
    // Method 1
    public void chargePhone() {
        System.out.println("Mengisi daya smartphone via port " + portType);
    }
    // Method 2
    public void activateFastCharge() {
        System.out.println("Status fast charge: " + isFastCharging);
    }
    // Method 3
    @Override
    public void printInfo() {
        System.out.println(x: "=== Info Charger HP ===");
        System.out.println("Brand         : " + brand);
        System.out.println("Wattage       : " + wattage + "W");
        System.out.println("Port Type     : " + portType);
        System.out.println("Fast Charging : " + isFastCharging + "\n");
    }
}
```

Gambar 27. Menambahkan 3 method di class charger Hp

- Class Charger Laptop sebagai turunan : chargeLaptop() Menghantarkan daya listrik tinggi khusus laptop, checkGrounding() Memeriksa keamanan arus arde/grounding pada bodi adaptor, printInfo() Mencetak rincian menyeluruh dari ukuran adaptor hingga tipe konektor.



```
chargerLaptop.java > chargerLaptop > chargerLaptop(String, int, String, String)
package P1;

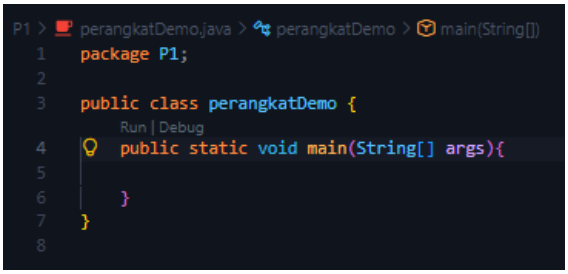
public class chargerLaptop extends charger {
    private String adapterSize;
    private String connectorType;

public chargerLaptop(String brand, int wattage, String adapterSize, String connectorType) {
        super(brand, wattage);
        this.adapterSize = adapterSize;
        this.connectorType = connectorType;
    }
    // Method 1
    public void chargeLaptop() {
        System.out.println("Mengalirkan daya ke laptop via port " + connectorType);
    }
    // Method 2
    public void checkGrounding() {
        System.out.println("Grounding adapter " + adapterSize + " aman.");
    }
    // Method 3
    @Override
    public void printInfo() {
        System.out.println(x: "=== Info Charger Laptop ===");
        System.out.println("Brand          : " + brand);
        System.out.println("Wattage        : " + wattage + "W");
        System.out.println("Adapter Size   : " + adapterSize);
        System.out.println("Connector Type : " + connectorType + "\n");
    }
}
```

Gambar 28. Menambahkan 3 method di class charger Laptop

g. Tambahkan satu class Demo sebagai main!
Jawaban :

- Class PerangkatDemo.java : Berfungsi sebagai kelas penguji yang memuat metode public static void main(String[] args) untuk menjalankan program secara terpadu.



```
P1 > perangkatDemo.java > perangkatDemo > main(String[])
1   package P1;
2
3   public class perangkatDemo {
        Run | Debug
4       public static void main(String[] args){
5
6       }
7   }
8
```

Gambar 29. Menambahkan class baru perangkatDemo.java

h. Instansiasikan satu buah objek untuk setiap class!
Jawaban :
Di dalam metode main, dibuat 5 objek nyata menggunakan kata kunci new:

- myLaptop: Objek konkret dari class Laptop dengan spesifikasi "Asus" dan 16 GB.
- myTws: Objek konkret dari class TWS dengan status "Soundcore" dan daya 90%.
- generalCharger: Objek konkret dari class Charger umum berdaya 65W.
- myHpCharger: Objek konkret dari class ChargerHp bertipe "Type-C" dengan *fast charging* aktif.
- myLaptopCharger: Objek konkret dari class ChargerLaptop berdaya 150W dengan konektor "Barrel".

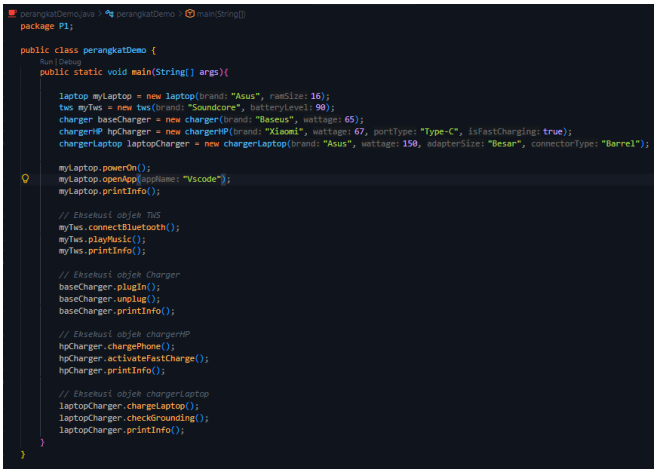


Gambar 30. Instansiasi 1 object untuk setiap class di perangkatdemo.java

i. Terapkan setiap method untuk setiap objek yang dibuat!
Jawaban :
Memanggil seluruh method yang dimiliki masing-masing objek secara berurutan:

- Menjalankan powerOn(), openApp(), dan printInfo() pada myLaptop.
- Menjalankan connectBluetooth(), playMusic(), dan printInfo() pada myTws.
- Menjalankan plugIn(), unplug(), dan printInfo() pada generalCharger.
- Menjalankan chargePhone(), activateFastCharge(), dan printInfo() pada myHpCharger.
- Menjalankan chargeLaptop(), checkGrounding(), dan printInfo() pada myLaptopCharger.

- code perangkatDemo.java



Gambar 31. Memanggil semua method yang dimiliki object

- Hasil

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PBO> javac P1/*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PBO> java P1.perangkatDemo
Laptop Asus berhasil dinyalakan.
Membuka aplikasi Vscode pada laptop.
=== Info Laptop ===
Brand : Asus
RAM   : 16 GB

TWS Soundcore terhubung ke perangkat.
Memutar audio melalui TWS.
=== Info TWS ===
Brand   : Soundcore
Battery : 90%

Charger Baseus terhubung ke listrik.
Charger Baseus dicabut dari listrik.
=== Info Charger ===
Brand   : Baseus
Wattage : 65W

Mengisi daya smartphone via port Type-C
Status fast charge: true
=== Info Charger HP ===
Brand         : Xiaomi
Wattage       : 67W
Port Type     : Type-C
Fast Charging : true

Mengalirkan daya ke laptop via port Barrel
Grounding adapter Besar aman.
=== Info Charger Laptop ===
Brand          : Asus
Wattage        : 150W
Adapter Size   : Besar
Connector Type : Barrel
```

Gambar 32. Hasil Run

3. Hasil dan Pembahasan

Berdasarkan eksekusi pada perangkatDemo.java, seluruh method berhasil dipanggil dan menghasilkan output teks yang sesuai tanpa runtime error.

Keterkaitan Konsep: Pewarisan (*Inheritance*) terbukti mengurangi redundansi kode. Class chargerHP dan chargerLaptop tidak perlu menulis ulang variabel brand dan wattage karena sudah ditangani oleh class charger. Penggunaan kata kunci super pada konstruktor turunan berhasil mengalirkan nilai argumen menuju variabel induk secara transparan

4. Kesimpulan

1. Pemrograman Berorientasi Objek (OOP) mempermudah pemodelan perangkat nyata ke dalam bentuk digital melalui keterpaduan data (atribut) dan perilaku (method).
2. Konsep pewarisan (*inheritance*) dengan kata kunci extends dan super meningkatkan efisiensi dan reusabilitas kode (*code reusability*) dengan mengelompokkan sifat-sifat umum pada sebuah kelas induk (*superclass*).
3. Penerapan modifier private dan protected memberikan kontrol akses yang aman terhadap integritas data internal objek (enkapsulasi).

5. Lampiran

Link Repository GitHub: https://github.com/Rappizr/PPBO/tree/main/P1